

APPLYING CRYPTOGRAPHY IN REAL - WORLD SCENARIO

Objectives

- Demonstrate data protection using AES-256 encryption
- Verify data integrity using SHA-256 hashing
- Implement secure messaging using RSA public key cryptography
- Demonstrate authenticity through digital signatures
- Simulate a real-word file verification
- Understand how cryptography protects modern information systems.

Tools Used

Tools	Purpose
Kali Linux	Operating Environment
OpenSSL 3.5.5	Cryptographic Operations
SHA-256	Hashing and Integrity
AES-256-CBC	Symmetric Encryption
RSA-2048	Public Key Cryptography

WHAT IS CRYPTOGRAPHY

Cryptography is the science of securing information by converting readable data into an unreadable form so only authorized parties can access it. Today, it sits quietly behind every bank transaction, WhatsApp message, email, software download, and countless other everyday interactions most people never think about.

DATA PROTECTION WITH ENCRYPTION

Step 1 - Create & encrypt the file

AES (Advanced Encryption Standard) is a symmetric encryption algorithm, meaning the same key locks and unlocks the data. We used AES-256-CBC here, which combines a 256-bit key with Cipher Block Chaining mode. That combination makes it extremely hard to crack through brute force.

```
(kali@PawnCipher)-[~]
$ cd techcrush_project

(kali@PawnCipher)-[~/techcrush_project]
$ echo "This is Group 30's confidential file.
This file is protected with AES encryption.
Unauthorized access is prohibited." > original_message.txt

(kali@PawnCipher)-[~/techcrush_project]
$ cat original_message.txt
This is Group 30's confidential file.
This file is protected with AES encryption.
Unauthorized access is prohibited.

(kali@PawnCipher)-[~/techcrush_project]
$ openssl enc -aes-256-cbc -salt -pbkdf2 -in original_message.txt -out encrypted_file.enc -pass pass:Group30@SecureKey

(kali@PawnCipher)-[~/techcrush_project]
$ od -c encrypted_file.enc | head -5
00000000  S a l t e d _ 264 257 225 351 006 366 9 U
00000020  033 001 242 z z 026 361 204 255 247 q 347 372 ( 361 J
00000040  ^ 240 321 \b s 361 214 221 260 m 365 177 375 < 271 315
00000060  x G 306 302 216 377 + 376 313 243 035 r 267 \a 256 +
00000100  274 316 337 # 340 303 177 347 K 177 375 201 244 325 I 271
```

Creating our plain text using the “echo” command and encrypting with AES-256-CBC encryption algorithm. The od -c output shows the cipher or encrypted text

Step 2 - Decrypt and verify

```
(kali@PawnCipher)-[~/techcrush_project]
$ openssl enc -aes-256-cbc -d -pbkdf2 -in encrypted_file.enc -out decrypted_message.txt -pass pass:Group30@SecureKey

(kali@PawnCipher)-[~/techcrush_project]
$ cat decrypted_message.txt
This is Group 30's confidential file.
This file is protected with AES encryption.
Unauthorized access is prohibited.
```

Successfully decrypted the cipher text back to its original plaintext

HASHING AND INTERGRITY CHECKING

A cryptographic hash function takes any input and turns it into a fixed-length string called a hash or digest. SHA-256 gives a 64-character hex output. What makes it powerful is the avalanche effect: even the tiniest change to the input produces a completely different hash, which makes it very good at catching tampering.

Step 1 - Hash the original, modify, hash again

```
(kali@PawnCipher)-[~/techcrush_project]
$ sha256sum original_message.txt
f3022e9df8b14e02ad3e614956e3dabf4c334a3e6dbc764e3efd1593bb5bc3a4  original_message.txt
```

Creating the hash value for the Text we created earlier(original_message.txt)

```
(kali@PawnCipher)-[~/techcrush_project]
$ echo "." >> original_message.txt

(kali@PawnCipher)-[~/techcrush_project]
$ sha256sum original_message.txt
f22682a2ff985fc27817e54fca7379c3ed7008b59f32ab08357d934de81e2f6c  original_message.txt
```

We altered the plain text just by adding "." To the original_message.txt and the whole hash value changes. Completely different hash, meaning tampering occurred.

PUBLIC KEY ENCRYPTION/SECURE MESSAGING

RSA is an asymmetric encryption algorithm built on a mathematically linked key pair. The public key encrypts data; only the matching private key can unlock it. This makes secure communication possible without both parties needing to share a secret in advance.

Step 1 - Generate key pair

```
(kali@PawnCipher)-[~/techcrush_project]
$ openssl genrsa -out private_key.pem 2048

(kali@PawnCipher)-[~/techcrush_project]
$ openssl rsa -in private_key.pem -pubout -out public_key.pem
writing RSA key

(kali@PawnCipher)-[~/techcrush_project]
$ ls -lh *.pem
-rw----- 1 kali kali 1.7K May 24 09:02 private_key.pem
-rw-rw-r-- 1 kali kali 451 May 24 09:02 public_key.pem
```

RSA-2048

key pair generated. The command "ls -lh *.pem" shows the permissions for both private key and public key. Notice the private key permissions are rw----- meaning only the owner can read it. The public key is rw-rw-r-- meaning anyone can read it.

Step 2 - View the public key

```
(kali@PawnCipher)-[~/techcrush_project]
$ cat public_key.pem
-----BEGIN PUBLIC KEY-----
MIIBIjANBgkqhkiG9w0BAQEFAAOCQAQ8AMIIBCgKCAQEA6vmjK0jBLLiZUDK4oc/0
q+p7ZbJbwrG06roY0pmIZ33u+3zHiMPtgIVDhb+t5PFAIzaJ5zuU7gx7yHE0kDD
2PaPowd9UHFTLY/1GfvxtbKbL0EGxe/ljLKZv3vr9bU/FUK9/UsWM7MSDoc5G/SU
pKFCiltt2uIIKdShnb06EJ980qJNLHks8SCJZ1sKVm8WyGWtpm276n+prxTvwYsw
9CpRy9joygx4n5V0bd4E1m1Cu2P9obbVlrr5Wgu0yXbUX2/1eE5NueiPtZd8bVB
zkV7g35H/5MfN5Wxkj2E0c2PFypn2000F7Zqh3rzo7fULXJD+sd8uYU0n8b+zCBw
/QIDAQAB
-----END PUBLIC KEY-----
```

Public key content and can be shared with anyone for **encrypting** messages.

Step 3 - Encrypt with public key

```
(kali@PawnCipher)-[~/techcrush_project]
$ echo "Hello Group 30! This message is encrypted with your public key. Only your private key can decrypt this." > secret_message.txt

(kali@PawnCipher)-[~/techcrush_project]
$ openssl pkeyutl -encrypt -inkey public_key.pem -pubin -in secret_message.txt -out rsa_encrypted.bin

(kali@PawnCipher)-[~/techcrush_project]
$ od -c rsa_encrypted.bin | head -5
0000000 218 177  - 365 273  C 243  0  : 312  W  |  a 313 233 027
0000020 254  j 833 324 006  e  j  z  z 361 264 200 265 322 314 004
0000040  r  U 373 355  |  j 207 237  R 004 251 323  o  A 314  0
0000060 233  M  : 036  P 201  j 251 257 247 323 206 351 223 203 224
0000100 227 315  j 243 251  k 260  +  d 310  t 321 201  ^ 005
```

Encrypting the secret_message.txt with the public key

Step 4 - Decrypt with private key

```
(kali@PawnCipher)-[~/techcrush_project]
$ openssl pkeyutl -decrypt -inkey private_key.pem -in rsa_encrypted.bin -out rsa_decrypted.txt

(kali@PawnCipher)-[~/techcrush_project]
$ cat rsa_decrypted.txt
Hello Group 30! This message is encrypted with your public key. Only your private key can decrypt this.

(kali@PawnCipher)-[~/techcrush_project]
$
```

We are using the private key to decrypt our secret_messahe.txt and it shows our real text here

Step 5 - Digital Signature: Verified OK

A digital signature proves that a file came from a specific sender and has not been changed since it was signed. The sender signs with their private key; anyone with the public key can verify it.

```
(kali@PawnCipher)-[~/techcrush_project]
$ openssl dgst -sha256 -sign private_key.pem -out signature.bin original_message.txt

(kali@PawnCipher)-[~/techcrush_project]
$ openssl dgst -sha256 -verify public_key.pem -signature signature.bin original_message.txt
Verified OK
```

File signed with private key and verified with public key and the result shows “Verified OK”

Step 6 - Signature fails on tampered file

```
(kali@PawnCipher)-[~/techcrush_project]
$ openssl dgst -sha256 -verify public_key.pem -signature signature.bin tampered_message.txt
Verification failure
40C7C795767F0000:error:02000068:rsa routines:ossl_rsa_verify:bad signature: ../crypto/rsa/rsa_sign.c:441:
40C7C795767F0000:error:1C800004:Provider routines:rsa_verify_directly:RSA lib: ../providers/implementations/signature/rsa_sig.c:1035:

(kali@PawnCipher)-[~/techcrush_project]
$ cat original_message.txt
This is Group 30's confidential file.
This file is protected with AES encryption.
Unauthorized access is prohibited.
.

(kali@PawnCipher)-[~/techcrush_project]
$ cat tampered_message.txt
This is a tampered file.
```

Verification failure signature does not match tampered file, proving authenticity protection. Hence the “Verification failure” result

Step 7 - Original vs tampered file

```
(kali@PawnCipher)-[~/techcrush_project]
$ cat original_message.txt
This is Group 30's confidential file.
This file is protected with AES encryption.
Unauthorized access is prohibited.
.

(kali@PawnCipher)-[~/techcrush_project]
$ cat tampered_message.txt
This is a tampered file.
```

Comparing the result of both the Original and Tampered file using the “cat” command

SCENARIO SIMULATION (FILE VERIFICATION SYSTEM)

When you download software, the publisher typically posts a SHA-256 hash on their site. You run the same hash on your downloaded file and compare the two. A match means it arrived clean. A difference means something changed in transit, most likely malware was slipped in.

For this demonstration, the actual Linux kernel README was downloaded from the official Linux

Torvalds GitHub repository.

Step 1 - Download a real file

```
(kali@PawnCIPHER)-[~/techcrush_project]
$ wget https://raw.githubusercontent.com/torvalds/linux/master/README -O downloaded_file.txt
--2026-05-24 09:34:12-- https://raw.githubusercontent.com/torvalds/linux/master/README
Resolving raw.githubusercontent.com (raw.githubusercontent.com)... 185.199.111.133, 185.199.110.133, 185.199.109.133, ...
Connecting to raw.githubusercontent.com (raw.githubusercontent.com)|185.199.111.133|:443 ... connected.
HTTP request sent, awaiting response... 200 OK
Length: 8843 (5.9K) [text/plain]
Saving to: 'downloaded_file.txt'

downloaded_file.txt                               100%[=====] 5.90K 6.83KB/s

2026-05-24 09:34:15 (6.83 KB/s) - 'downloaded_file.txt' saved [8843/8843]

(kali@PawnCIPHER)-[~/techcrush_project]
$ cat downloaded_file.txt
Linux kernel

The Linux kernel is the core of any Linux operating system. It manages hardware,
system resources, and provides the fundamental services for all other software.

Quick Start
-----
* Report a bug: See Documentation/admin-guide/reporting-issues.rst
* Get the latest kernel: https://kernel.org
* Build the kernel: See Documentation/admin-guide/quickly-build-trimmed-linux.rst
* Join the community: https://lwn.kernel.org/

Essential Documentation
-----

All users should be familiar with:

* Building requirements: Documentation/process/changes.rst
* Code of Conduct: Documentation/process/code-of-conduct.rst
* License: See COPYING

Documentation can be built with make htmldocs or viewed online at:
https://www.kernel.org/doc/html/latest/

Who Are You?
-----
```

wget downloads the official Linux kernel README from GitHub — real-world file.

Step 2 - Generate and save the hash

```
(kali@PawnCIPHER)-[~/techcrush_project]
$ sha256sum downloaded_file.txt
2844b0b2cafe22741724c4fdda79b1259de48743f12ad40ce408adb1ef00ceda downloaded_file.txt

(kali@PawnCIPHER)-[~/techcrush_project]
$ sha256sum downloaded_file.txt > original_hash.txt

(kali@PawnCIPHER)-[~/techcrush_project]
$ cat original_hash.txt
2844b0b2cafe22741724c4fdda79b1259de48743f12ad40ce408adb1ef00ceda downloaded_file.txt
```

Two hash values generated, one from the website and the other from our kali device to check its originality. And both hashes are the same, meaning the website is safe

Step 3 - Inject malware and verify


```
(kali@PawnCipher)-[~/techcrush_project]
$ echo "MALWARE INJECTED BY ATTACKER" >> downloaded_file.txt

(kali@PawnCipher)-[~/techcrush_project]
$ sha256sum -c original_hash.txt
downloaded_file.txt: FAILED
sha256sum: WARNING: 1 computed checksum did NOT match
```

MALWARE INJECTED into the file or website downloaded, sha256sum -c returns FAILED.
Tampering detected.

Step 4 - Re-download clean file and verify

```
(kali@PawnCipher)-[~/techcrush_project]
$ wget https://raw.githubusercontent.com/torvalds/linux/master/README -O downloaded_file.txt
--2026-05-24 09:51:09-- https://raw.githubusercontent.com/torvalds/linux/master/README
Resolving raw.githubusercontent.com (raw.githubusercontent.com)... 185.199.108.133, 185.199.111.133, 185.
Connecting to raw.githubusercontent.com (raw.githubusercontent.com)|185.199.108.133|:443... connected.
HTTP request sent, awaiting response... 200 OK
Length: 6043 (5.9K) [text/plain]
Saving to: 'downloaded_file.txt'

downloaded_file.txt                                100%[=====]

2026-05-24 09:51:11 (44.6 KB/s) - 'downloaded_file.txt' saved [6043/6043]

(kali@PawnCipher)-[~/techcrush_project]
$ sha256sum -c original_hash.txt
downloaded_file.txt: OK
```

Re- downloaded Clean file sha256sum -c returns OK. File is safe and unmodified.

WHAT EACH CRYPTOGRAPHIC METHOD PROTECTS FROM THE PROJECT

DATA PROTECTION WITH ENCRYPTION

Protects: CONFIDENTIALITY

Confidentiality means only the right people can read the data. In this project, we created a plaintext file using a simple echo command, encrypted it with AES-256-CBC, and inspected the output using **od -c**, it came back as a mess of random-looking bytes. That is confidentiality doing exactly what it should.

AES-256 uses one secret key to both lock and unlock information. With 2^{256} possible key combinations, brute-force is not a realistic attack with today's computing power. Once the file was encrypted, the contents were meaningless to anyone without the key, whether that person intercepted it mid-transfer or pulled it from storage.

Real-World Application:

Banks use this to protect account data sitting on their servers. WhatsApp uses it to encrypt stored media on your phone. AWS and Google Cloud use it on backup files. The note we made

during the project, that keeping the secret key actually secret is everything, is not a minor observation. It is the whole point of symmetric encryption. The algorithm holds. Where things fall apart is in how people handle the key.

HASHING AND INTEGRITY CHECKING

Protects: INTEGRITY

Integrity means being able to tell if data has been touched, even slightly. This project showed that clearly. We added a single period to the file and the new hash looked nothing like the original. One character, completely different output. It is a simple test but the results are hard to argue with.

Hashing works differently from encryption. You cannot reverse a hash to get back to the original file. Its job is not to hide data but to fingerprint it. SHA-256 produces the same 64-character output format whether the input is a single letter or a 10GB video. Change anything in the file, even one bit, and roughly half the output bits' flip. That is the Avalanche Effect, and it is what makes hashing useful for integrity checks.

Real-World Application:

Package managers like apt and pip use this exact process every time you install software. Each package has a published SHA-256 hash, and the system checks your download against it. If they do not match, the install stops. Companies also use this to catch tampering in audit logs, database backups, and legal digital evidence. Our single-period test was a small-scale version of what happens when malware slips into a legitimate download.

PUBLIC KEY ENCRYPTION/SECURE MESSAGING

Protects: AUTHENTICITY

Authenticity is about proving two things: who sent something, and that the content has not changed since they sent it. We tested both through the RSA key pair and the digital signature steps.

On the encryption side, using the public key to encrypt and the private key to decrypt keeps the message confidential in transit. The signature side is just as important. Signing the file with the private key and verifying with the public key came back "Verified OK," which confirmed two things at once: the file came from whoever holds that private key, and nothing in the file had been touched since signing. When we made a small change to the file and tried to verify again, it failed immediately. The signature broke on contact with even the smallest edit.

Keeping the private key locked down (only readable by the owner) while the public key is open to anyone is what makes RSA so useful here. Anyone can verify a signature using the public key, but only the person holding the private key can actually produce one.

Real-World Application:

Software publishers sign their release packages so users can check the software actually came from them and was not tampered with. SSL/TLS certificates use the same logic to authenticate websites. When a browser shows a padlock, it has verified an RSA or ECDSA signature chain. Governments use digital signatures on identity documents, and banks use them to authorize high-value transactions.

FILE VERIFICATION SYSTEM

Protects: INTEGRITY

For this deliverable, we used an actual file from a real source rather than something we created ourselves. We downloaded the Linux kernel README straight from Linus Torvalds’ GitHub repository using wget. Its hash matched. We then injected a line into the file to simulate malware, and the system flagged it as FAILED. After re-downloading the clean file, it returned OK.

What this protected was integrity at the moment of delivery, right when a file moves from a trusted source to a user’s device. That is exactly where real malware attacks happen. An attacker intercepts a legitimate download and inserts code before it reaches the end user. Hash verification catches that immediately, no matter how small or subtle the change. The exercise made clear why you should always verify downloads rather than assume they arrived clean.

Real-World Application:

Operating system update systems, antivirus signature databases, and financial software distributors all use this method. It also sits at the core of blockchain technology. Every block holds the hash of the one before it, so altering any past record breaks the entire chain from that point forward.

Summary

Deliverable	Method	Protects	Real-World Example
1	AES-256-CBC	Confidentiality	Bank data at rest, WhatsApp media, cloud backups
2	SHA-256 Hashing	Integrity	apt/pip package verification, audit logs
3	RSA-2048 + Digital Signature	Authenticity	SSL certificates, software code signing, digital IDs
4	File Verification System	Integrity in Delivery	OS updates, malware detection, blockchain

CRYPTOGRAPHIC TECHNIQUES USED

DATA PROTECTION WITH ENCRYPTION

This section covered symmetric encryption using AES-256-CBC. Symmetric means the same key is used to both encrypt and decrypt. AES-256-CBC pairs a 256-bit key with Cipher Block Chaining, which links each encrypted block to the previous one so identical input blocks

produce different cipher text output. That is why AES-256 has become the global standard for protecting data at rest.

We created a plaintext file using the **echo** command with a short message identifying it as Group 30's confidential file, then encrypted it using OpenSSL's **enc -aes-256-cbc -salt -pbkdf2** flags. The **-pbkdf2** flag is important: it tells OpenSSL to use Password-Based Key Derivation Function 2 to stretch the passphrase into a proper 256-bit key before using it, rather than using the raw passphrase directly. This significantly strengthens the encryption against dictionary attacks.

After encryption, the output was inspected using the **od -c** command, which displays the raw byte content. The result was exactly what good cipher text should look like: a dense, unreadable stream of random-appearing bytes with no trace of the original message. The file began with the bytes for the string **Salted**, which is OpenSSL's indicator that a salt value was included a critical step that ensures even if the same file is encrypted twice with the same password, the output is different each time.

Decryption was performed using the same key and the **enc -d** flag. The output was compared against the original using **diff**, which produced no output confirming an exact byte-for-byte match. Decryption succeeded perfectly.

HASHING AND INTEGRITY CHECKING:

This deliverable demonstrated the use of cryptographic hashing for data integrity verification. A hash function is fundamentally different from encryption: it is one-way, meaning the hash cannot be reversed to recover the original data. Its purpose is not to hide information but to fingerprint it. SHA 256 always produces a 64-character hexadecimal digest, regardless of whether the input is a single character or a 10-gigabyte file.

The SHA-256 hash of the original message file was generated using the **sha256sum** command. This produced the first hash value, a specific 64-character string that represents the exact byte-for-byte fingerprint of the file at that moment.

A deliberate, minimal modification was then made to the file: a single period was appended to the end using **echo "." >> original_message.txt**. The file was hashed again. The resulting 64-character string was completely different not similar, not slightly changed, but entirely unrecognizable as having any relationship to the first hash. This is the Avalanche Effect: a defining mathematical property of all strong hash functions where a single-bit input change causes approximately half of all output bits to flip.

PUBLIC KEY ENCRYPTION AND SECURE MESSAGING:

This deliverable introduced asymmetric cryptography through RSA-2048, the most widely deployed public-key cryptographic system in the world. Unlike AES, RSA uses two mathematically linked keys: a public key that can be shared with anyone and a private key that

must never leave the owner's possession. Data encrypted with the public key can only be decrypted with the corresponding private key, and vice versa.

A 2048-bit RSA key pair was generated using **openssl genrsa -out private_key.pem 2048**, followed by extracting the public key with **openssl rsa -in private_key.pem -pubout -out public_key.pem**. Immediately after generation, the file permissions were checked using **ls -lh *.pem**. The private key showed permissions of **rw-----**, meaning only the file owner can read it. The public key showed **rw-rw r--**, meaning it is readable by anyone. This permission structure is not optional in a production environment it is mandatory.

A secret message was then encrypted using the public key with **openssl pkeyutl -encrypt -inkey public_key.pem -pubin**. The encrypted output was inspected with **od -c | head -5** and confirmed to be completely unreadable binary data. Decryption using the private key recovered the original message exactly.

The digital signature component was where the real power of asymmetric cryptography became apparent. The original message file was signed using the private key with SHA-256 as the digest algorithm. Signature verification using the public key returned "Verified OK" — confirming simultaneously that the file came from the private key holder and that it had not been altered since signing. The file was then deliberately modified and verification was attempted again. The result was an immediate and explicit "Verification failure" error with a full RSA signature mismatch trace. The tampered file comparison using **cat** showed the contrast clearly: the original contained the legitimate message while the tampered file contained a single replacement line.

REAL-WORLD FILE VERIFICATION SYSTEM

This deliverable applied SHA-256 hashing to a genuine real-world scenario, using an actual file from an actual source rather than locally created test data. The Linux kernel README was downloaded directly from the official Linus Torvalds GitHub repository using the **wget** command. This is a publicly available, authoritative file a real-world equivalent of downloading software from a trusted publisher.

After the download completed, **sha256sum** was run against the file and the hash was saved to a reference file called **original_hash.txt**. The hash was verified against itself using **sha256sum -c**, which returned "OK" confirming that the file arrived intact and unmodified from the source.

The attack simulation then began. A line of text was injected into the file using the **echo** command to represent malware or unauthorized code insertion. **sha256sum -c** was run again against the saved reference hash. The result was unambiguous: "FAILED: 1 computed checksum did NOT match." The system correctly detected the injection without knowing what was injected, without reading the file content, and without any signature purely by comparing hash values.

Finally, the clean version of the file was re-downloaded using **wget** and **sha256sum -c** was run

one more time. The result returned to **nothing** confirming that the original, unmodified file matches the stored hash and is safe.

CHALLENGES FACED AND HOW THEY APPLY TO REAL-WORLD SYSTEMS

Practical Challenges Encountered During the Project

Working through this project on Kali Linux with OpenSSL 3.5.5 threw up a few real problems that ended up being some of the most useful parts of the exercise.

- **Command-line precision:** Before any cryptography started, a typo caused the first issue. Typing “mkdir techcrush” with a space between the words created two separate folders instead of one. In cryptographic environments, that kind of mistake can quietly send key files or output to the wrong location and cause failures that are hard to trace back.
- **OpenSSL version compatibility:** The “rsautl” command, which shows up in a lot of older guides, is deprecated in OpenSSL 3.5.5. We had to switch to “pkeyutl” for RSA operations. It is a good reminder that cryptographic tools keep changing, and relying on outdated commands can produce unexpected results or fail with no clear error message.
- **Key management responsibility:** Generating the RSA-2048 key pair was not technically difficult, but it raised something worth sitting with. If the private key is lost, every file encrypted with the corresponding public key is gone permanently. There is no recovery option.

Application to Real-World Systems

Symmetric Encryption (AES-256-CBC); The AES-256-CBC encryption from the first project is the same standard banks, governments, AWS S3, and Google Cloud use to protect stored data and backups. That said, in this project we used a human-readable passphrase rather than a raw binary key, which is a common shortcut but not a secure one..

- ✓ AES normally needs a 128, 192, or 256-bit binary key. When a password is supplied instead, OpenSSL uses PBKDF2 (via the -pbkdf2 flag) to derive a proper key from it. That strengthens things, but the password itself is still the weakest point if it ever shows up in command history.
- ✓ Production systems handle this differently. They use Hardware Security Modules (HSMs) or dedicated key management platforms like AWS KMS or HashiCorp Vault to generate, store, and rotate AES keys without ever exposing them in plaintext to anyone.

Asymmetric Encryption (RSA-2048); The RSA key pair from the third project highlights a practical limitation worth knowing. RSA cannot encrypt files larger than its key size minus

padding. With a 2048-bit key, that ceiling is roughly 245 bytes.

✓ To get around this, production systems use hybrid encryption. A random AES key encrypts the large file, and RSA encrypts only that small AES key. HTTPS, PGP email encryption, and secure file transfer protocols all work this way.

✓ File permissions are critical: the “private_key.pem” generated in this project showed permissions of rw----- (only owner can read). In production, `chmod 600 private.pem` must be applied immediately after generation. A key readable by other users negates all cryptographic protection.

Hashing and File Verification (SHA-256); The file verification scenario showed that SHA-256 picks up tampering immediately. One injected line was all it took to produce a completely different hash. But there is a real vulnerability worth flagging in how this approach is commonly deployed.

✓ The Risk: An attacker who modifies a file on a server can also just regenerate the checksum file to match. Running “`sha256sum new_compromised_file > checksum.txt`” updates the reference, and the tampered file now passes verification.

✓ The Fix: Proper deployment systems like AIDE sign the checksum repository with an offline RSA private key, or store hashes on a separate read-only logging server. That is hashing and digital signatures working together, which is exactly what projects two and three combined.

Conclusion;

The math behind AES, SHA-256, and RSA is solid. That was not where the problems showed up in this project. The weak points were human: how keys are stored, which commands are used, how output is handled, and how carefully the environment is set up. Command history leaks, process visibility, swap exposure, and randomness quality are the kinds of things that break real systems, not the algorithms themselves. The tools we used here, OpenSSL on Kali Linux, are the same one’s security professionals use in the field. What we ran into during this project maps directly onto what comes up in enterprise security operations, penetration testing, and secure system design.